

CONTEÚDO TEÓRICO - AULA 06

Disciplina: Robótica Móvel Inteligente

Tema: Infraestrutura de Software para Robótica Industrial

1. Por que usar o ROS 2 e não um código tradicional?

Quando começamos a programar, aprendemos a criar scripts **monolíticos**: um único arquivo de código que executa passo a passo.

Para robôs simples isso funciona. Mas imagine um Robô Cão-Guia movimentando em uma calçada:

- Ele precisa ler o LiDAR em tempo real (10 vezes por segundo).
- Precisa analisar imagens da câmera com Visão Computacional para identificar o sinal do semáforo.
- Precisa calcular a rota no mapa para não trombar com as pessoas.
- Precisa tocar um áudio de voz para o usuário.

Se colocarmos tudo isso em um único código tradicional, quando a câmera demorar 0,5 segundo para processar uma imagem, o robô vai "travar", deixar de ler o LiDAR e bater em um obstáculo!

O **ROS 2 (Robot Operating System)** não é um sistema operacional como o Windows ou Linux, mas um middleware "gerenciador de tráfego" de softwares. Em vez de um código gigante, dividimos o robô em vários programas pequenos e independentes chamados Nós (*Nodes*).

Pense no ROS 2 como uma equipe médica em cirurgia. Você não tem um único médico fazendo a anestesia, cortando, medindo a pressão e monitorando os batimentos. Você tem vários especialistas (**Nós**) trabalhando ao mesmo tempo. Eles se comunicam (**Mensagem**) dando dados essenciais no centro cirúrgico ou por rádio (**Tópicos**).

- **Nó (Node):** Um pequeno programa focado em apenas *uma* tarefa.
- **Tópico (Topic):** Um canal de comunicação.
- **Mensagem (Message):** O formato do dado transmitido (ex: velocidade, coordenadas, imagem da câmera).
- **Publisher (Publicador) e Subscriber (Inscrito):** O Nó do LiDAR *publica* as distâncias no tópico **/scan**. O Nó de Navegação se *inscreve* no tópico **/scan** para escutar essa informação e decidir para onde ir.

2. A Camada de Infraestrutura: Linux, WSL2 e Docker

Por que o Linux é o padrão da Robótica Mundial?

Robôs precisam de **tempo real** e **baixo nível**. O Linux permite que os programas conversem direto com a memória e os processadores sem que o sistema operacional decida "pausar" seu código para rodar uma atualização de segundo plano (comum no Windows). Quase toda a biblioteca de robótica industrial e visão computacional avançada é desenvolvida e otimizada nativamente para **Ubuntu Linux**.

O Papel do WSL2 / WSLg no Windows

O **WSL2 (Windows Subsystem for Linux 2)** roda um Kernel do Linux *real* lado a lado com o Windows.

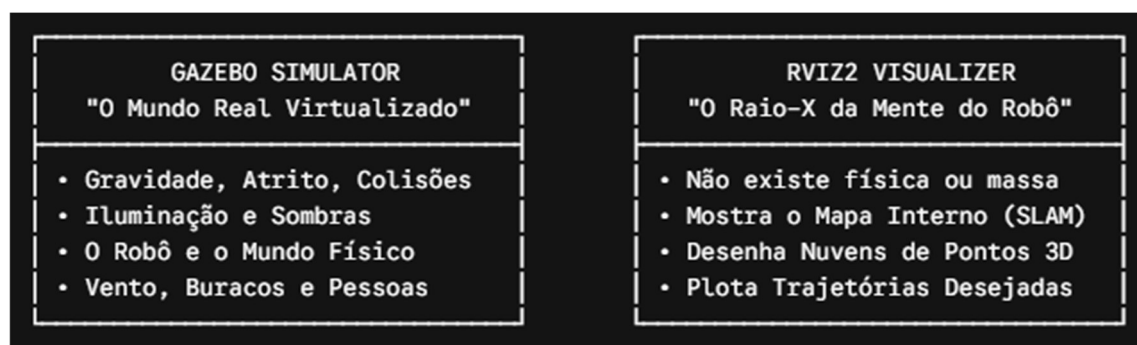
- **WSLg (WSL Graphics):** É o módulo que permite que o Linux "pegue emprestada" a placa de vídeo (GPU) do Windows. Isso é crucial para abrirmos janelas de simulação 3D sem que o computador vire um "forno" ou trave em 2 quadros por segundo.

O Papel do Docker (Containers)

O **Docker** cria a "embalagem" (chamada de Container). Ele empacota o ROS 2, as bibliotecas, o Python e os drivers gráficos na versão exata que precisamos. Assim, a frase *"na minha máquina funciona, mas na sua não"* deixa de existir.

3. Simulação e Telemetria: Entendendo a diferença entre Gazebo e RViz2

Por que eu preciso de duas janelas 3D abertas ao mesmo tempo?



Gazebo (Simulador)

O **Gazebo** é o "Video Game" do robô. Nele recriamos a física comum:

- Se o robô acelerar demais em uma rampa, ele tomba.
- Se bater em um poste, a colisão o faz parar.
- Os sensores simulados (câmera, LiDAR) reagem à luz e ao relevo do ambiente digital.
- Propósito: Testar o robô sem correr o risco de quebrar um protótipo físico real de milhares de reais.

RViz2 (Visualizador de Dados e Telemetria)

O **RViz2 (ROS Visualization)** é a "Ferramenta de Raio-X". Ele não simula física, ele apenas desenha o que o robô está *pensando e enxergando*:

- Ele mostra as linhas do laser do LiDAR batendo nas paredes.
- Ele desenha o mapa preto e branco que o robô está construindo na memória.
- Ele mostra a rota planejada em verde na tela.
- Propósito: Descobrir o porquê de o robô ter tomado uma decisão errada. Se o robô bater no Gazebo, olhamos para o RViz2 para ver se ele enxergou o obstáculo ou se estava "cego".

4. Conectando teoria com a prática:

DOG-IA - Robô Cão-Guia Urbano de Inteligência Assistiva.

Entender essa base teórica é o que permitirá construir a arquitetura do robô:

1. O **Gazebo** vai simular a calçada, o semáforo e os obstáculos aéreos.
2. O **ROS 2** vai integrar a câmera, o laser e os motores em microsserviços.
3. O **RViz2** vai nos mostrar o robô mapeando a cidade em tempo real.
4. O **Docker/WSL2** garantirá que todos trabalhem com o mesmo ambiente calibrado.